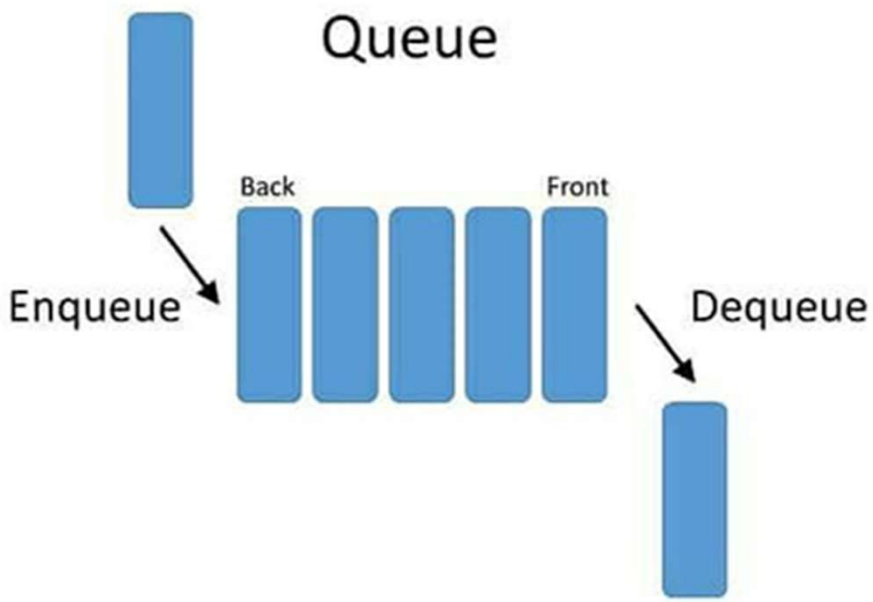
BFS和queue

115/3/6

講師:167904呂宥丞

queue

與list列表不同，是採「先進先出」原則



舉例

```
from collections import deque
queue = deque( )
queue.append("A")
queue.append("B")
queue.append("C")
print(queue.popleft( ))  # 輸出: "A" (最早添加的元素)
print(queue)  # 輸出: deque(["B", "C"])
```

BFS

BFS是queue的一種應用，可以用來遍歷樹(如下圖)的每一個元素，並取得樹直徑或是樹上每個元素間的距離等

```
    A
   / \
  B   C
 / \
D   E
```

舉例

```
from collections import deque

graph = { 'A': ['B', 'C'], 'B': ['D', 'E'], 'C': [ ], 'D': [ ], 'E': [ ] }

def  bfs(start):
        queue = deque([start])  # 建立一個只有start的queue佇列
        visited = set( )  #紀錄已訪問過的元素

        while queue:
                node = queue.popleft( )
                if node not in visited:
                        print(node)
                        visited.add(node)  #在visited中記錄node
                        queue.extend(graph[node])  #在queue中放此枝節下的元素
bfs('A')
```

舉例

```python
from collections import deque
graph = {'A':['B','C'],'B':['D','E'],'C':[],'D':[],'E':[]}
def bfs(start):
    queue = deque([start])
    dist = {start: 0} # 起點距離為 0
    while queue:
        node = queue.popleft()
        for neighbor in graph[node]:
            if neighbor not in dist: # 沒走過
                dist[neighbor] = dist[node] + 1
                queue.append(neighbor)
    return dist
print(bfs('A'))
```

隨堂練習

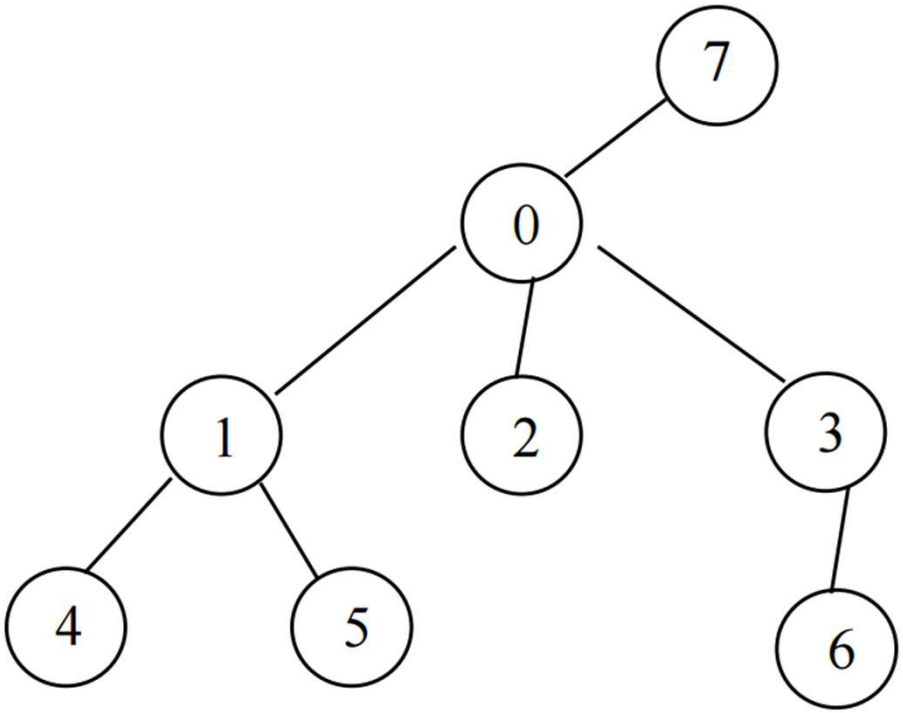
b967.4.血緣關係

小宇有一個大家族。有一天，他發現記錄整個家族成員和成員間血緣關係的家族族譜。
小宇對於最遠的血緣關係 (我們稱之為"血緣距離") 有多遠感到很好奇。

右圖為家族的關係圖。
0 是 7 的孩子， 1、2 和 3 是 0 的孩子， 4 和 5 是 1 的孩子， 6 是 3 的孩子。
我們可以輕易的發現最遠的親戚關係為 4(或 5) 和 6 ，他們的"血緣距離"是 4 (4→1→0→3→6)。

給予任一家族的關係圖，請找出最遠的"血緣距離"。

你可以假設只有一個人是整個家族成員的祖先，而且沒有兩個成員有同樣的小孩。



隨堂練習

輸入說明

第一行為一個正整數 N 代表成員的個數，每人以 0~N-1 之間唯一的編號代表。

接著的 N-1 行，每行有兩個以一個空白隔開的整數 a 與 b (0 ≤ a, b ≤ N-1)，代表 b 是 a 的孩子。

輸出說明:

輸出一行，輸出最遠"血緣距離"的答案。

解答

```python
from collections import deque
n=int(input())
family=[[] for _ in range(n)]
for _ in range(n-1):
    a,b=map(int,input().split())
    family[a].append(b)
    family[b].append(a)

def bfs(start):
    dist=[-1]*n
    dist[start]=0
    q=deque([start])
    while q:
        u=q.popleft()
        for p in family[u]:
            if dist[p]==-1:
                dist[p]=dist[u]+1
                q.append(p)
    f=start
    for i in range(n):
        if dist[i]>dist[f]:
            f=i
    return f,dist[f]

a,_=bfs(0)
b,dis=bfs(a)
print(dis)
```

先找到其中一個端點

再找離此端點最遠的點

兩點距離即為最遠的「血緣距離」

謝謝大家

非常感謝大家願意聽我分享
這是我第一次做程式教學簡報
如有任何不足之處或覺得能補充的
歡迎大家提出，謝謝大家